CIVICTRACK STUDY HANDBOOK & ARCHITECTURE NOTES

How Login, Registration & Role-Based Dashboards Work

A complete, simple, and detailed guide designed for learning, coding comprehension, and explaining confidently in project reviews, viva, and technical interviews.

💡 Module 0: The Simple Real-World Analogy

If someone asks: *"How does role-based access and authentication work in your civic issue application?"*, explain it using the **"Hospital / Airport Card System"** analogy:

🏢 THE SMART BUILDING ANALOGY

Imagine a smart municipal building:

- **The Entrance (Login Page):** Everyone walks through the same front door. You show your digital ID card (Email & Password).
- **The Smart Badge (Role Check):** Once scanned, the security system detects your badge level:
  - **Resident Badge (`citizen`):** You are guided to the Public Citizen Service Desk. You can report potholes or streetlights and view your own submitted tokens.
  - **Commissioner Badge (`admin`):** The elevator takes you directly to the 10th Floor Executive Command Center. You see city-wide stats, all complaints, and assign contractors.
  - **Contractor Badge (`staff`):** You are directed to the Field Operations

> Workshop. You only see tickets assigned to your repair crew, and upload photos once repairs are finished.

Module 1: The Three System Roles Explained

ROLE: CITIZEN

Citizen Resident

General public residents living in city wards (e.g., Arun Kumar, Priya Sharma).

- Registers complaints on map
- Uploads proof photos & location
- Receives live tracking code (e.g. `TRK-2026-104`)
- Views **only their own** complaints
- Reopens issues if repairs are unsatisfactory

ROLE: ADMIN

Municipal Administrator

Municipal officers & commissioners (e.g., Commissioner Rajesh V.).

- City-wide master triage queue
- Assigns tickets to contractors
- Enrolls & manages vendor staff
- Reviews AI duplicate complaints
- Analyzes geospatial hotspot clusters

ROLE: STAFF

Maintenance Contractor

Field maintenance repair agencies (e.g., Apex Infra, CityClean).

- Sees work orders assigned to their crew
- Pins job locations on field ward map
- Updates status to `in_progress`
- Uploads **after-repair proof photo**
- Closes ticket with completion notes

🔐 Module 2: How the Login Process Works (Code Decode)

Login starts at `src/pages/public/LoginRegister.jsx` and is processed inside `src/context/AuthContext.jsx`.

1 User enters Email and Password

User fills out the login form and clicks *"Sign In to Account"*.

2 Form submit calls `login(email, password)`

React intercepts the submit with `e.preventDefault()` and sends credentials to `AuthContext`.

3 Validation against LocalStorage Users Array

`AuthContext` reads all registered users from `Storage.getUsers()` and checks email & password match.

4 State & Storage Persistence

If valid, it sets `currentUser` in React state and saves to `Storage.setCurrentUser(user)` in `localStorage`.

5 Dynamic Role-Based Redirect

The UI checks the user's role and navigates to the exact dashboard meant for them.

Code Breakdown: The Redirection Function

```
// In LoginRegister.jsx:
const redirectAfterLogin = (userRole) => {
  if (userRole === 'admin') {
    navigate('/admin');     // Directs to Municipal Command Center
  } else if (userRole === 'staff') {
    navigate('/staff');     // Directs to Vendor Field Portal
  } else {
    navigate('/citizen');   // Directs to Citizen Self-Service Portal
```

```
  }
};
```

> 🎓 **VIVA / INTERVIEW QUESTION: "WHY DOES THE USER STAY LOGGED IN WHEN THEY REFRESH THE BROWSER?"**
>
> **Answer:** "Because we synchronize React state with `localStorage`. On initial load, inside an `useEffect` in `AuthContext.jsx`, we call `Storage.getCurrentUser()`. If a user was previously logged in, their session is automatically rehydrated into React memory, avoiding unwanted logouts."

📝 Module 3: How Registration Works

Registration allows any visitor to create an account as a Citizen, Maintenance Contractor, or Administrator.

The 5 Steps of Registration:

1. **Select Account Type:** User chooses between *Citizen*, *Maintenance Contractor*, or *Municipal Official*.
2. **Enter Account Info:** Full Name, Email, Password (with eye show/hide toggle), Phone number, and Address / Department.
3. **Email Duplication Check:** `AuthContext` checks if any existing user already has this email:

   ```
   if (allUsers.some((u) => u.email.toLowerCase() === cleanEmail)) {
     throw new Error('An account with this email already exists.');
   }
   ```

4. **New User Object Creation:** A unique timestamp ID (`Date.now()`) is assigned with a default avatar.

5. **Automatic Login & Dispatch:** The new user is saved to `localStorage`, marked as `currentUser`, and immediately redirected to their new dashboard!

📊 Module 4: How the Dashboards Work Based on Roles

1. Citizen Dashboard (`/citizen`)

CITIZEN SELF-SERVICE

Core Concept: User Isolation. A citizen must never see another citizen's private complaints.

```
// In CitizenDashboard.jsx:
const { currentUser } = useAuth();
const { issues } = useIssues();

// ISOLATION FILTER: Only issues submitted by THIS citizen
const myIssues = issues.filter((i) => i.user_id === currentUser?.user_id);
```

Key Features:

- **Clickable KPI Summary Cards:** Total Reports, In Progress, Resolved, Pending. Clicking any card filters the view live.
- **Report New Issue Button (`/report`):** Opens the complaint registration form with GPS location picker and live AI duplicate checker.
- **Personal Reports Archive (`/citizen/my-reports`):** Full list of all historical submissions with live timeline logs.

2. Admin Command Center (`/admin`)

MUNICIPAL COMMAND

Core Concept: City-Wide Aggregation & Contractor Allocation. The administrator sees every ticket across the entire municipality.

```
// In AdminDashboard.jsx:
const unreviewedIssues = issues
  .filter((i) => ['reported', 'under_review'].includes(i.status));
```

```
const urgentHazards = issues
  .filter((i) => i.priority === 'urgent' && i.status !== 'resolved');
```

Key Features:

- **Dedicated Left Sidebar (`AdminSidebar`):** Sticky sidebar with live badge count for triage, vendors, and duplicates.
- **Contractor Workload Real-Time Monitor:** Shows how many active jobs each vendor currently has to avoid overloading a single contractor.
- **Master Triage Queue (`/admin/reports`):** Allows triaging any complaint, changing status, or dispatching to a vendor.
- **Vendor Management (`/admin/vendors`):** Add/edit contracting companies, assign trade departments, and generate contractor codes.
- **Section 17 AI Deduplication (`/admin/duplicates`):** Calculates match scores between complaints using:
  `Duplicate Score = 0.4 × Location + 0.3 × Category + 0.3 × Keywords`
- **Section 18 Geospatial Hotspots (`/admin/hotspots`):** Cluster visualization identifying geographic complaint density within 450 meters.

3. Vendor / Staff Dashboard (`/staff`)

FIELD CONTRACTOR PORTAL

Core Concept: Work Order Execution & Proof Submission. Contractors receive dispatches and submit repair proof.

```
// In StaffDashboard.jsx:
// Contractor only sees issues specifically assigned to their contractor ID
const assignedIssues = issues.filter(
  (i) => i.assigned_to === currentUser?.user_id
);
```

The Resolution Process (How a Contractor closes a ticket):

1. Contractor reviews assigned work order on their interactive ward map.
2. Crew goes to the site and completes the repair (pothole asphalt patch, streetlight LED replacement, drainage unclogging).
3. Clicks **"Update Work Order"** button on the ticket.
4. Changes status to `resolved`.
5. Attaches an **after-repair photographic proof** and enters work notes.
6. Saves: Ticket becomes resolved, citizen is automatically notified!

🔄 Module 5: The Complete Lifecycle of an Issue

Status	Who Performs It?	What Happens?
`reported`	Citizen	Issue created via `/report` with GPS coordinates, photos, and tracking code.
`under_review`	Admin	Municipal officer opens triage queue, verifies complaint authenticity.
`assigned`	Admin	Admin selects contractor (e.g. Apex Infra) and sends work order.
`in_progress`	Vendor	Contractor arrives on-site and marks work started.
`resolved`	Vendor	Contractor uploads completion photo and notes; ticket closed.
`reopened`	Citizen / Admin	If repair is substandard, citizen reopens with reason; sends back to contractor.

🎯 Module 6: Quick Cheat Sheet for Presentations & Viva

Q: What is the Single Source of Truth?

`AuthContext` and `IssueContext` manage global React state, and automatically synchronize with `window.localStorage` so data remains

Q: How do you prevent layout collisions?

The top `Navbar` dynamically inspects `currentUser.role`. For Admins and Staff, center links are hidden because their navigation is neatly organized in `AdminSidebar` and `StaffSidebar`.

consistent across all components and page reloads.

Q: How did you fix the page scroll issue?

We created a universal `ScrollToTop` component that sets `window.history.scrollRestoration = 'manual'` and resets `window.scrollTo(0, 0)` on every route change, preventing pages from opening at the bottom.